

Optimal Networks and Best Routes - Final of Unit 5

Lecture 20

Data Structures and Algorithms

Where We Left Off

Last lecture, we learned how to **model** real-world networks as graphs and how to **explore** them systematically using DFS and BFS.

But our traversals treated all edges equally. In the real world, edges carry **weights** -- distances, costs, travel times. Two critical questions arise:

1. **"What is the cheapest way to connect everything?"** -- Minimum Spanning Trees
2. **"What is the shortest route from A to B?"** -- Shortest Path Algorithms

Today we answer both. These algorithms power GPS navigation, network infrastructure, airline routing, and more.

Part 1: Spanning Trees

What is a Spanning Tree?

Given a **connected, undirected graph** $G = (V, E)$, a **spanning tree** is a subgraph T that:

- 1. Includes **all** vertices of G
- 2. Is a **tree** -- connected and contains no cycles
- 3. Has exactly $|V| - 1$ edges

A single graph can have **many** different spanning trees. Consider a triangle (3 vertices, 3 edges): removing any one edge produces a valid spanning tree. So the triangle has 3 possible spanning trees.

Original:	Spanning Tree 1:	Spanning Tree 2:	Spanning Tree 3:
A --- B	A --- B	A --- B	A B
/			/
/			/
/			/
C	C	C	C

What is a Minimum Spanning Tree (MST)?

When the graph is **weighted**, each spanning tree has a total weight (the sum of its edge weights). The **Minimum Spanning Tree** is the spanning tree with the **smallest total weight**.

Why Does This Matter?

Real-world optimization problems:

- A **telecom company** wants to connect n cities with fiber-optic cable at minimum cost
- An **electrical engineer** must wire n circuit components using the least amount of wire
- A **municipality** wants to build roads connecting n villages as cheaply as possible

In each case, the MST gives the **cheapest way to keep everything connected** without redundant links.

Key Properties

- MST has exactly $|V| - 1$ edges (it is a tree)
- MST may **not be unique** if multiple edges share the same weight
- Removing any MST edge disconnects the graph; adding any non-MST edge creates a cycle

The Graph We Will Use

We will trace both Prim's and Kruskal's algorithms on the same weighted, undirected graph to see how two very different strategies arrive at the same optimal answer.

2

(A)------(B)

| \

6| \ 3 |5

| \

(C) (D)

\ |

9 \ 1|

\ |

(F) (E)

\ /

8 \ /

*

Edge List:

A--B : 2

A--D : 3

B--E : 7

D--E : 1

E--F : 8

D--F : 10

A--C : 6

B--D : 5

C--F : 9

E--C : 4

C--F : 9

6 vertices (A-F), 10 edges. Let us find the MST (which will have 5 edges).

Part 1a: Prim's Algorithm

Prim's Algorithm: The Idea

Prim's algorithm grows the MST **one vertex at a time**, starting from any single vertex and always adding the cheapest edge that reaches a new vertex.

The Greedy Strategy

1. Start with any vertex. It is the initial MST (a tree of 1 node).
2. Look at all edges that connect an **MST vertex** to a **non-MST vertex**.
3. Pick the edge with the **smallest weight**.
4. Add that edge and the new vertex to the MST.
5. Repeat until all vertices are included.

This is a **greedy** algorithm -- at each step, we make the locally optimal choice (cheapest available edge). The remarkable mathematical fact: this locally optimal strategy produces the globally optimal MST.

Prim's Algorithm: Step-by-Step Trace

Starting from vertex **A**:

Step	MST Vertices	Candidate Edges (MST to non-MST)	Chosen Edge	Cost
0	{A}	A-B(2), A-C(6), A-D(3)	A-B (cheapest)	2
1	{A, B}	A-C(6), A-D(3), B-D(5), B-E(7)	A-D (cheapest)	3
2	{A, B, D}	A-C(6), B-E(7), D-E(1), D-F(10)	D-E (cheapest)	1
3	{A, B, D, E}	A-C(6), E-C(4), E-F(8), D-F(10)	E-C (cheapest)	4
4	{A, B, D, E, C}	E-F(8), D-F(10), C-F(9)	E-F (cheapest)	8

MST Edges: A-B(2), A-D(3), D-E(1), E-C(4), E-F(8)

MST Total Weight: $2 + 3 + 1 + 4 + 8 = 18$

Prim's Algorithm: Visualizing the Growth

Step 0: Start at A
(A)

Step 1: Add B (cost 2)
(A)---2---(B)

Step 2: Add D (cost 3)
(A)---2---(B)
 \
 3 \
 (D)

Step 3: Add E (cost 1)
(A)---2---(B)
 \
 3 \
 (D)
 |
 1|
 |
 (E)

Step 4: Add C (cost 4)
(A)---2---(B)
 \
 3 \
 (D)
 |
 1|
 |
 (E)---4---(C)

Step 5: Add F (cost 8)
(A)---2---(B)
 \
 3 \
 (D)
 |
 1|
 |
 (E)---4---(C)
 |
 8|
 |
 (F)

At each step, the tree grows by exactly one vertex and one edge. After 5 steps, all 6 vertices are connected.

Prim's: Why Does Greedy Work Here?

This seems almost too good to be true. Why does always picking the cheapest edge give the optimal solution?

The Cut Property (Intuition):

Consider any "cut" that divides the vertices into two groups: those already in the MST and those not yet included. The cheapest edge crossing this cut **must** be in the MST.

Why? If we used a more expensive crossing edge instead, we could swap it for the cheaper one, reducing the total weight -- contradicting the claim that we had the minimum spanning tree.

This is the formal guarantee that Prim's greedy choice is always safe.

Complexity

- With a simple array to track minimum edges: $O(|V|^2)$ -- good for dense graphs
- With a **min-heap** (priority queue): $O((|V| + |E|) \log |V|)$ -- good for sparse graphs

Remember binary heaps from our Heaps unit? This is exactly where they shine as priority queues.

Part 1b: Kruskal's Algorithm

Kruskal's Algorithm: A Different Strategy

While Prim's grows the MST from a single vertex outward, Kruskal's takes a **global** view: consider all edges at once, sorted by weight, and greedily add the cheapest edges that do not create a cycle.

The Algorithm

1. **Sort** all edges by weight (lightest first)
2. For each edge in sorted order:
 - If adding this edge **does not create a cycle**, add it to the MST
 - If it **would create a cycle**, skip it
3. Stop when we have $|V| - 1$ edges

The key question: how do we efficiently check whether adding an edge creates a cycle?

The Cycle Detection Problem: Union-Find

Kruskal's relies on a clever data structure called **Disjoint Set** (or **Union-Find**).

The Idea

- Initially, every vertex is in its own "set" (its own mini-component)
- When we add an edge (u, v) , we **merge** (union) the sets containing u and v
- Before adding an edge, we check: are u and v already in the **same set**?
 - **Same set?** Adding this edge would create a cycle -- skip it
 - **Different sets?** Safe to add -- merge their sets

Operations

- **Find(x)** : Which set does vertex x belong to?
- **Union(x, y)** : Merge the sets containing x and y

With **path compression** and **union by rank** optimizations, both operations run in nearly $O(1)$ time (technically $O(\alpha(n))$, where α is the inverse Ackermann function -- effectively constant).

Kruskal's Algorithm: Step-by-Step Trace

Step 1: Sort all edges by weight:

#	Edge	Weight
1	D--E	1
2	A--B	2
3	A--D	3
4	E--C	4
5	B--D	5
6	A--C	6
7	B--E	7
8	E--F	8
9	C--F	9
10	D--F	10

Kruskal's Algorithm: Processing Edges

Step 2: Process each edge. Add it if it does not create a cycle.

Edge	Weight	Sets Before	Action	Sets After
D--E	1	{A} {B} {C} {D} {E} {F}	D and E in different sets. Add.	{A} {B} {C} {D,E} {F}
A--B	2	{A} {B} {C} {D,E} {F}	A and B in different sets. Add.	{A,B} {C} {D,E} {F}
A--D	3	{A,B} {C} {D,E} {F}	A in {A,B}, D in {D,E}. Different. Add.	{A,B,D,E} {C} {F}
E--C	4	{A,B,D,E} {C} {F}	E in {A,B,D,E}, C in {C}. Different. Add.	{A,B,C,D,E} {F}
B--D	5	{A,B,C,D,E} {F}	B and D both in same set. Skip (cycle!).	--
A--C	6	{A,B,C,D,E} {F}	Same set. Skip.	--
B--E	7	{A,B,C,D,E} {F}	Same set. Skip.	--
E--F	8	{A,B,C,D,E} {F}	Different sets. Add.	{A,B,C,D,E,F}

Stop! We have 5 edges ($|V| - 1 = 6 - 1$). MST is complete.

Kruskal's: The Result

MST Edges: D-E(1), A-B(2), A-D(3), E-C(4), E-F(8)

MST Total Weight: $1 + 2 + 3 + 4 + 8 = 18$

This is the **same MST and same total weight** as Prim's! Two completely different strategies, same optimal answer. This is guaranteed by the theory: for a graph with unique edge weights, the MST is unique.

Final MST:

```
(A)---2---(B)
  \
   3 \
      (D)
      |
      1|
      |
      (E)---4---(C)
      |
      8|
      |
      (F)
```

Edges skipped (B-D, A-C, B-E, C-F, D-F) would all have created cycles in the growing forest.

Prim's vs. Kruskal's: Direct Comparison

Feature	Prim's	Kruskal's
Strategy	Grow the MST from one vertex	Process edges globally (cheapest first)
Cycle avoidance	Implicitly -- only considers edges to new vertices	Uses Union-Find to detect cycles
Data structure	Priority Queue (min-heap)	Disjoint Set (Union-Find)
Needs sorting?	No	Yes (sort all edges first)
Complexity (with good DS)	$O((V + E) \log V)$	$O(E \log E)$
Best for	Dense graphs ($ E \approx V ^2$)	Sparse graphs ($ E \ll V ^2$)
Starts with	A single vertex	Individual edges

Both are greedy. Prim's is "vertex-centric" (grow from a point). Kruskal's is "edge-centric" (pick the best edges globally). Both rely on the Cut Property for correctness.

Part 2: Shortest Path -- Dijkstra's Algorithm

The Shortest Path Problem

Given a **weighted graph** and a **source vertex** s , find the shortest path from s to every other vertex.

"Shortest" means **minimum total edge weight** along the path -- not fewest edges.

Recall: BFS finds the shortest path by **edge count** (unweighted graphs). But if edges have different weights, BFS fails:

BFS says A→C is shortest (1 edge): But the weighted shortest path is A→B→C:

A ---10--- C

(BFS: 1 edge, weight 10)

```

  \      /
1 \    / 2
   \  /
    B

```

A ---1--- B ---2--- C

(Dijkstra: 2 edges, weight 3)

BFS picks 1 edge of weight 10. Dijkstra picks 2 edges of total weight 3.

Constraint: All edge weights must be **non-negative** ($w(u, v) \geq 0$ for all edges).

Where Is This Used?

Dijkstra's algorithm is everywhere:

- **GPS Navigation:** Find the fastest route from your location to a destination city
- **Network Routing:** Internet packets find the lowest-latency path between servers
- **Airline Systems:** Determine the cheapest flight path with layovers
- **Game AI:** Characters find the shortest walkable path around obstacles
- **Social Networks:** Find the "closest connection" between two people weighted by interaction strength

Every time your phone gives you driving directions, a variant of Dijkstra's algorithm is working behind the scenes.

Dijkstra's Algorithm: The Idea

Dijkstra's is another **greedy** algorithm. It maintains a **distance table** -- the best known distance from the source to every other vertex.

The Core Mechanism: Relaxation

When we discover that going through vertex u gives a **shorter** path to vertex v than the one we currently know, we update our records. This is called **relaxation**:

```
if dist[u] + weight(u, v) < dist[v]:  
    dist[v] = dist[u] + weight(u, v)    // We found a shorter path!
```

Think of it like discovering a highway shortcut: "I thought the best route to city v was 50 km, but by going through city u first, I can get there in only 35 km. Update the map!"

Dijkstra's Algorithm: Step by Step

The Algorithm

1. **Initialize:** Set $dist[s] = 0$, $dist[v] = \infty$ for all other vertices. All vertices are "unvisited."
2. **Repeat** until all vertices are visited:
 - a. Select the unvisited vertex u with the **smallest** $dist[u]$
 - b. Mark u as **visited** (its shortest distance is now finalized)
 - c. For each unvisited neighbor v of u :
 - **Relax:** If $dist[u] + w(u, v) < dist[v]$, then update $dist[v] = dist[u] + w(u, v)$
3. When done, $dist[v]$ contains the shortest distance from s to v for every vertex v .

Why it works (greedy correctness): When we pick the unvisited vertex with the smallest distance, that distance is **final** -- no shorter path can possibly be found later. This is because all remaining edges are non-negative, so any alternative path through unvisited vertices can only be longer.

The Graph for Dijkstra's

We will trace Dijkstra's on this **directed, weighted** graph with source = **A**:



Goal: Find the shortest distance from A to every other vertex.

Dijkstra's: Traced Example

Initial State: $\text{dist} = \{A:0, B:\infty, C:\infty, D:\infty, E:\infty, F:\infty\}$. All unvisited.

Step 1: Visit A (dist = 0, smallest unvisited)

Relax neighbors of A:

- $\text{dist}[B] = \min(\infty, 0 + 4) = 4$ -- update!
- $\text{dist}[C] = \min(\infty, 0 + 2) = 2$ -- update!

Vertex	A	B	C	D	E	F
dist	0	4	2	∞	∞	∞
visited	Y					

Step 2: Visit C (dist = 2, smallest unvisited)

Relax neighbors of C:

- $dist[D] = \min(\infty, 2 + 3) = 5$ -- update!
- $dist[E] = \min(\infty, 2 + 1) = 3$ -- update!

Vertex	A	B	C	D	E	F
dist	0	4	2	5	3	∞
visited	Y		Y			

Step 3: Visit E (dist = 3, smallest unvisited)

Relax neighbors of E:

- $dist[F] = \min(\infty, 3 + 6) = 9$ -- update!

Vertex	A	B	C	D	E	F
dist	0	4	2	5	3	9
visited	Y		Y		Y	

Step 4: Visit B (dist = 4, smallest unvisited)

Relax neighbors of B:

- $dist[D] = \min(5, 4 + 1) = 5$ -- no change (equal, not shorter)
- $dist[E] = \min(3, 4 + 3) = 3$ -- no change (E already has a shorter path)

Vertex	A	B	C	D	E	F
dist	0	4	2	5	3	9
visited	Y	Y	Y		Y	

Step 5: Visit D (dist = 5, smallest unvisited)

Relax neighbors of D:

- $\text{dist}[F] = \min(9, 5 + 2) = 7$ -- update! *Found a shorter path to F through D!*

Vertex	A	B	C	D	E	F
dist	0	4	2	5	3	7
visited	Y	Y	Y	Y	Y	

Step 6: Visit F (dist = 7, smallest unvisited)

No unvisited neighbors. Done!

Final shortest distances from A: A:0, B:4, C:2, D:5, E:3, F:7

Tracing Back the Shortest Paths

The distance table tells us **how far**, but not **which route**. To reconstruct the actual path, we track the **predecessor** (the vertex that last updated each distance):

Vertex	Shortest Dist	Predecessor	Shortest Path from A
A	0	--	A
B	4	A	A → B
C	2	A	A → C
D	5	B or C	A → C → D or A → B → D
E	3	C	A → C → E
F	7	D	A → C → D → F

Notice vertex F: initially we found the path $A \rightarrow C \rightarrow E \rightarrow F$ (cost 9), but Step 5 discovered a shorter path $A \rightarrow C \rightarrow D \rightarrow F$ (cost 7). This is relaxation in action -- improving our estimate when we find a better route.

Dijkstra's: Why Negative Weights Break It

Dijkstra's assumes that once a vertex is visited (finalized), no shorter path to it can ever be found. This is true only when edge weights are non-negative.

1

A -----> B

\ /

2 \ / -5

v v

C

Dijkstra from A: visit B(1), then C(2). Final: dist[C] = 2.

But the actual shortest path is A -> B -> C = 1 + (-5) = -4.

The negative edge breaks the greedy assumption.

With the negative edge $B \rightarrow C$ of weight -5 , going through B actually reaches C with distance -4 , which is cheaper than the direct edge of weight 2. But Dijkstra finalized C at distance 2 before discovering this.

For graphs with negative weights, use the **Bellman-Ford** algorithm instead ($O(|V| \cdot |E|)$, slower but handles negatives correctly).

Dijkstra's: Complexity and Connections

Time Complexity

- With a **simple array** to find the minimum: $O(|V|^2)$ -- better for dense graphs
- With a **min-heap** (binary heap priority queue): $O((|V| + |E|) \log |V|)$ -- better for sparse graphs

Connection to Prim's Algorithm

Dijkstra's and Prim's are structurally very similar:

	Prim's MST	Dijkstra's Shortest Path
Goal	Minimize edge weight to add	Minimize total distance from source
Key value	Weight of cheapest edge to reach v	Shortest distance from s to v
Greedy choice	Pick vertex closest to the MST	Pick vertex closest to the source
Data structure	Min-heap	Min-heap

Both use a priority queue, both process one vertex at a time, and both rely on greedy correctness. If you understand one, you are very close to understanding the other.

Interactive Visualization Tool

For seeing Prim's, Kruskal's, and Dijkstra's algorithms animate step-by-step, try this excellent tool:

VisuAlgo -- Graph Algorithms:

<https://visualgo.net/en/mst> (for MST algorithms)

<https://visualgo.net/en/sssp> (for Dijkstra's shortest path)

You can build your own graph, set edge weights, and watch the algorithm run step by step. The visualization highlights which vertex is being processed, which edges are being relaxed, and how the solution builds over time.

Part 3: The Complete Unit Summary

Hash Tables and Graphs -- The Full Picture

Hash Tables & Graphs Unit (Lectures 16-20):

- |
- +-- Hash Tables (Lectures 16-18)
 - | +-- Hashing concept, hash functions (division, multiplication, strings)
 - | +-- Hash table structure, separate chaining, open addressing
 - | +-- Linear probing, quadratic probing, double hashing
 - | +-- Tombstones and clustering problems
 - | +-- Load factor, rehashing, amortized $O(1)$
 - | +-- Applications: dictionaries, symbol tables, caches, sets
- +-- Graphs (Lectures 19-20)
 - +-- Graph terminology, types (directed, undirected, weighted)
 - +-- Representations: adjacency matrix vs adjacency list
 - +-- Traversals: DFS (stack), BFS (queue)
 - +-- Connected components
 - +-- Minimum Spanning Trees: Prim's, Kruskal's
 - +-- Shortest Path: Dijkstra's algorithm

Key Complexity Summary

Algorithm / Structure	Time Complexity	Space
Hash Table (average case)	$O(1)$ insert / search / delete	$O(n)$
Hash Table (worst case)	$O(n)$ insert / search / delete	$O(n)$
DFS / BFS	$O(V + E)$	$O(V)$
Prim's MST (with heap)	$O((V + E) \log V)$	$O(V)$
Kruskal's MST	$O(E \log E)$	$O(V)$
Dijkstra's (with heap)	$O((V + E) \log V)$	$O(V)$

Notice the pattern: heaps (priority queues) appear in Prim's and Dijkstra's. Stacks power DFS. Queues power BFS. Linked lists power chaining. **Every data structure we studied earlier becomes a building block for these advanced algorithms.**

Connecting All the Dots

The data structures we have studied across this entire course form a progression:

Structure	Key Operation	Used By
Arrays	$O(1)$ random access	Hash tables (bucket storage), adjacency matrix
Linked Lists	$O(1)$ insert/delete at head	Chaining in hash tables, adjacency lists
Stacks	LIFO push/pop	DFS traversal
Queues	FIFO enqueue/dequeue	BFS traversal
Binary Heaps	$O(\log n)$ extract-min	Priority queues in Prim's and Dijkstra's
BSTs	$O(\log n)$ ordered operations	When sorted access matters (hash tables trade this away)
Hash Tables	$O(1)$ average lookup	Visited sets, frequency counting, symbol tables
Graphs	Model any relationship	Networks, maps, dependencies

Each new structure builds on the ones before. This is the architecture of computer science.

What Comes Next?

You now have a solid foundation in the core data structures and algorithms of computer science. Here are some directions for further study:

Shortest Paths:

- Bellman-Ford algorithm (handles negative edge weights)
- Floyd-Warshall (all-pairs shortest paths in $O(|V|^3)$)

Graph Algorithms:

- Topological sorting (scheduling, build systems)
- Strongly connected components (Kosaraju's, Tarjan's)
- A* search (Dijkstra's + heuristics -- used in game pathfinding)

Advanced Data Structures:

- Red-Black Trees, AVL Trees (self-balancing BSTs)
- Tries (efficient string storage)
- B-Trees (database indexing)

